学习航线图

CORE MISSION

A.手把手搭建双节点vLLM推理集群——从裸机到跨节点推理的完整实操导航

如何将两台独立的物理主机（主机A和主机

探索历程

搭建舞台：先备好两台'厨房'

想象你要开一家双厨餐厅。你不可能只有一个灶台就开张——你需要两个厨房（两台主机），每个厨房都要有完整的炉具（WSL Ubuntu环境）和通风管道（虚拟交换机桥接网络）。只有两个厨房都能独立运转，才能开始考虑让它们协同做菜。

B1. 环境准备：双主机WSL与虚拟交换机配置确认

在主机A和主机B上分别确认WSL Ubuntu系统可用，并通过get vm switch命令检查虚拟交换机的桥接模式配置，确保两台主机之间网络互通，为后续集群通信奠定基础。

主厨先就位：Head节点启动并'等菜'

双厨餐厅里，先让主厨（Head节点）上岗。主厨先把自己的工具摆好（安装依赖、设置环境变量），启动流水线（vLLM服务），但他不会马上做菜——因为菜谱要求两个厨师同时到场才能开火。于是他站在门口，等着副厨（Worker节点）到来。

B2. HEAD节点启动：依赖安装与VLLM服务初始化

在主机A的WSL Ubuntu中安装必要依赖，执行start_head.sh脚本，启动Head节点。该脚本包含环境变量配置、Head节点启动命令及vLLM服务启动。此时vLLM配置为流水线并行模式，因集群为双节点，服务会阻塞等待Worker节点加入后才开始加载模型。

副厨报到：拿着地址找到主厨

副厨（Worker节点）到了。他需要知道主厨在哪个厨房（Head节点的IP地址），然后走过去报到。副厨也有自己的

B3. WORKER节点启动：连接HEAD节点并加入RAY集群

工具箱（worker脚本），到达后通过Ray集群的调度系统与主厨握手。

在主机B上同样确认WSL环境和虚拟交换机配置后，进入WSL Ubuntu系统，执行start_worker.sh脚本，指定Head节点的IP地址（如4.0.0.18）进行连接。通过ray status命令可验证两个节点均已成功加入集群，两个GPU均被识别。

- **双厨合璧：同步菜谱，同时开火**

两位厨师都到位了，接下来要把菜谱（模型信息）同步给两个人。主厨把菜谱传给副厨，两人同时开始备菜（加载模型）。你能从两个厨房的监控屏幕上看到，两边的灶台（GPU显存）都在慢慢升温——说明他们正在同步工作。

B4. 模型加载与同步：跨节点流水线并行

Head节点接收到Worker节点后，将模型信息同步至Worker节点，两个节点同时开始加载模型。通过Ray dashboard（Head节点IP:8625端口）可观察两个节点的GPU显存逐渐升高，模型从Head节点拉取并同步加载，直至vLLM服务状态变为started/completed。

- **上菜验证：客人点单，双厨协作出餐**

餐厅开门迎客了。客人在前台（Gradio WebUI）点了一道菜（输入问题），订单先传到主厨（Head节点），主厨协调副厨一起完成这道菜。你能从监控屏上看到两个灶台同时升温（GPU利用率上升），菜越多（输出文本越长），两个灶台就越忙。菜出完了，灶台自然就凉下来了。

B5. 推理验证与集群监控：端到端测试与GPU利用率观察

通过Gradio脚本（修改IP地址和端口号8000）启动WebUI，输入问题触发推理。Head节点接收请求后协调Worker节点共同进行文本生成（decode过程）。通过Ray dashboard实时观察两个节点GPU利用率的变化：推理时利用率上升，文本越长占用越高，推理结束后利用率逐渐回落。

B1. 环境准备：双主机WSL与虚拟交换机配置确认

引入

章前总览： 在正式搭建 vLLM 分布式推理集群之前，必须先解决一个最基础的物理层问题——**两台独立主机之间能否在网络层面"看见"对方？** 如果主机 A 和主机 B 的 WSL 子系统之间无法互通，后续的 Ray 集群组建、Head/Worker 节点通信、模型参数同步都将无从谈起。 因此，本章的核心任务是：**验证双主机 WSL 环境可用，并确认虚拟交换机的桥接网络配置正确，为跨节点通信铺平道路。**

逻辑链条：

- 确认 WSL 子系统可用：如何进入主机 A / 主机 B 的 WSL Ubuntu 系统？ → **(C1)**
- 确认虚拟交换机桥接配置：WSL 的网络是如何与外部打通的？如何检查配置是否正确？ → **(C2)**
- 确认网络互通：确保后续 Head 节点 IP（如 4.0.0.18）可被 Worker 节点访问。

> 只有 **C1** 和 **C2** 均确认无误，才能保证 **B2**（Head 节点启动）和 **B3**（Worker 节点连接）之间的网络通信不会成为瓶颈。

C1. WSL 命令与 Ubuntu 子系统进入方式

WSL (Windows Subsystem for Linux) 是微软在 Windows 10/11 上提供的兼容层技术。

它并非传统虚拟机，而是通过 **Windows 内核的系统调用翻译层** 来运行 Linux 二进制可执行文件（ELF 格式）。

AI 补充

WSL 有两个主要版本，它们的底层实现差异直接决定了本场景的技术选型：

WSL 1 · 翻译层

系统调用实时翻译为 Windows NT 内核调用

✓ 启动快 · 资源占用低

✗ 不支持完整内核特性（如 Docker 原生运行）

WSL 2 · 轻量虚拟机

基于 Hyper-V + 完整 Linux 内核

✓ 兼容性强（Docker、GPU 直通）

✗ 启动稍慢 · 网络配置更复杂

本场景结论：vLLM 需要 GPU 直通和分布式网络通信，因此**必然使用 WSL 2**。

操作步骤

1 进入 WSL 子系统

在 Windows 的 PowerShell 或 CMD 中，输入以下命令即可进入默认的 WSL 发行版：

```
wsl
```

如需指定发行版，可使用 `wsl -d <发行版名称>` 。

2 **验证子系统可用性**

进入 Ubuntu 后，确认当前环境正常：

```
uname -a
```

若输出中包含 **Linux ... microsoft-standard-WSL2**，说明当前运行在 WSL 2 环境下，符合后续 vLLM 部署要求。

在 Windows 命令中误将 `wsl` 写成了大写的 `WSL`（在部分旧版 CMD 中大小写敏感），或拼写错误如 `swl`、`wl`，会导致命令无法识别。**请务必使用小写 `wsl`**。

C2. 虚拟交换机桥接配置与网络互通确认

在确认 WSL 2 可用之后，下一步要解决的是**网络连接问题**。WSL 2 由于运行在轻量虚拟机中，其网络默认通过 **NAT（网络地址转换）** 模式与 Windows 主机通信，这种模式下外部设备无法直接访问 WSL 内部服务。要实现主机 A 与主机 B 的 WSL 之间互相通信，必须通过 **虚拟交换机（vSwitch）桥接模式** 将 WSL 网络桥接到物理网络。

认知路径：从 NAT 到桥接

默认 NAT 模式 → 外部不可见 → 桥接到 vSwitch → 跨主机互通

检查步骤

1 查看虚拟交换机列表

在 Windows PowerShell（管理员权限）中执行：

```
Get-VMSwitch
```

该命令会列出当前系统中所有虚拟交换机及其类型。

2 确认桥接类型

在输出结果中，重点关注 **SwitchType** 这一列。本场景要求 WSL 使用 **External（外部桥接）** 类型，使 WSL 内的网络与物理网卡处于同一网段。

✓ **正确配置**

Host A / Host B 的 WSL 都能 ping 通对方 IP

✗ **异常情况**

ping 超时或返回"目标主机不可达"

3 查看 WSL 网卡信息

进入 WSL Ubuntu 后，查看本机网络地址：

```
ip addr show
```

确认 WSL 获取到的 IP 地址与主机处于**同一子网**（例如主机 A IP 为 4.0.0.x，WSL 也应落在 4.0.0.x 网段）。

2

台独立主机（Host A / Host B）

1

个共享网段（如 4.0.0.0/24）

- **WSL 2 默认 NAT 模式不可跨主机通信**，必须将虚拟交换机设置为 External（桥接模式）。
- 修改 vSwitch 后，**需要重启 WSL**（`wsl --shutdown`）才能使网络配置生效。
- 两台主机的 vSwitch 必须桥接到**同一物理网络**，否则 WSL 之间无法互通。

本节小结

- 环境准备阶段的核心任务：**确认双主机 WSL 子系统可用，并完成虚拟交换机桥接配置**，为跨节点通信奠定网络基础。
- vLLM 需要 GPU 直通和分布式通信，因此必须使用 **WSL 2**（而非 WSL 1），通过 `wsl` 命令即可进入 Ubuntu 子系统。
- 使用 `Get-VMSwitch` 检查虚拟交换机类型，确保为 **External（桥接）**，并让两台主机的 WSL 处于同一子网、能够互相 ping 通。
- 只有 C1、C2 均确认无误，才能保证后续 **B2（Head 节点启动）** 和 **B3（Worker 节点连接）** 的网络通信不出问题。

B2. Head节点启动：依赖安装与vLLM服务初始化

引入

章前总览： 在完成双主机WSL环境与虚拟交换机配置确认（B1）之后，我们已确保主机A和主机B之间的网络互通。本章的任务是在主机A上正式启动集群的“大脑”——**Head节点**。核心问题是：**如何正确初始化Head节点，并理解它为何在启动后并不立即加载模型，而是进入“阻塞等待”状态？**

逻辑链条：

- C1. 如何进入WSL环境并确认虚拟交换机配置 → `start_head.sh`脚本的结构与作用
- C2. Head节点的IP在WSL环境中意味着什么 → local IP与WSL网络环境
- C3. 为什么vLLM启动后不加载模型而是等待 → 流水线并行与多节点集群的启动条件

C1. start_head.sh脚本的结构与作用

在主机A的WSL Ubuntu系统中，我们需要安装必要依赖并执行Head节点启动脚本。该脚本是集群启动的核心入口，其内部结构可以分为三大模块。

模块 1 **环境变量配置** → 模块 2 **Ray集群初始化** → 模块 3 **vLLM服务启动**

1 **环境变量配置**

设置 `MASTER_ADDR` 、 `MASTER_PORT` 、 `WORLD_SIZE` 、 `RANK` 等关键通信参数。

2 **Ray集群初始化**

使用 `ray start --head` 命令将当前节点设为 Ray 集群的 Head 节点。

3 **vLLM服务启动**

指定模型路径、并行策略（如流水线并行 `pipeline_parallel_size=2` ）、监听端口等参数。

执行流程

1. 在 WSL Ubuntu 中安装依赖（如 Ray、vLLM、PyTorch 等）
2. 执行 `bash start_head.sh`
3. 脚本依次完成：设置环境变量 → 启动 Ray Head 节点 → 启动 vLLM 服务

💡 AI 补充

要理解为什么启动脚本中必须包含 Ray 组件，需要先看清 vLLM 的分布式推理架构：它并不是自己实现节点通信，而是依赖 Ray 提供底层的调度能力。

> 💡 vLLM 的分布式推理依赖 **Ray** 作为底层分布式计算框架

> 🔗 Ray 负责节点发现、任务调度与跨节点通信

Head节点在Ray集群中扮演“协调者”角色——它维护全局状态、接收Worker注册、分发任务。因此启动 Ray Head 节点，是 vLLM 分布式推理的**前置条件**。

> 🎯 没有 Ray 集群，vLLM 无法感知 Worker 节点的存在

依赖安装顺序：务必先安装 Ray，再安装 vLLM。vLLM 的分布式模块在导入时会检查 Ray 是否可用，顺序颠倒将导致导入失败。

环境变量填错：`MASTER_ADDR` 必须填写 Head 节点在 WSL 中的 IP，而非 Windows 主机的 IP。这是初学者最容易混淆的地方。

C2. local IP与WSL网络环境

在 `start_head.sh` 中有一个关键变量 **local IP**，即 Head 节点的 IP 地址。这个 IP 是 WSL 虚拟环境内部的 IP，而非 Windows 主机的 IP。

NAT 模式（默认）

- WSL 拥有独立的虚拟网卡
- IP 通常为 `172.x.x.x` 网段
- 属于内部私有 IP
- 外部主机**无法直接访问**

桥接模式（本案例）

- WSL 获得与主机同网段的 IP
- 如 `4.0.0.18`
- 在局域网中表现为独立主机
- **可被其他主机访问** ✓

为何必须使用 WSL 内部 IP?

Ray 集群的节点间通信发生在 **WSL 实例之间**。Worker 节点（主机 B 的 WSL）需要通过这个 IP 连接到 Head 节点（主机 A 的 WSL）。

如果填写 Windows 主机的 IP，Worker 节点将**无法正确路由**到 WSL 内部的 Ray 进程。

💡 AI 补充

知道了为什么要用 WSL 内部 IP 之后，下一步就是如何准确获取这个 IP 以及验证网络是否连通。

🔍 获取 IP：执行 `ip addr show` 或 `hostname -I`

📌 本案例 Head 节点 IP 为 `4.0.0.18`

验证方法：在主机 B 的 WSL 中执行以下命令，若能连通则说明网络配置正确：

⚡ `ping 4.0.0.18` → 有响应即连通

NAT 与桥接混淆：NAT 模式下 WSL 的 IP 是内部私有 IP，外部主机无法直接访问；桥接模式下 WSL 获得独立局域网 IP，可被其他主机访问。本案例**必须使用桥接模式**，否则 Worker 节点无法连接。

IP 变动问题：WSL 重启后 IP 可能变化。建议在 `start_head.sh` 中动态获取 IP（如 `hostname -I | awk '{print $1}'`），而非硬编码。

C3. 流水线并行与多节点集群的启动条件

当 `start_head.sh` 执行后，vLLM 服务启动，但此时 GPU 显存并未被占用，服务处于 **阻塞等待** 状态。这是因为 vLLM 配置了流水线并行，且集群为双节点，必须等待 Worker 节点加入后才能开始加载模型。

流水线并行（Pipeline Parallelism）

将模型按层切分成多个阶段，每个阶段部署在不同的节点/GPU 上。前向传播时数据依次经过各阶段，反向传播时梯度依次回传。



本案例中 `pipeline_parallel_size=2`，表示模型被切分为 **2 个阶段**。

为何必须等待 Worker 节点?

1. vLLM 启动时根据 `pipeline_parallel_size` 和 `world_size` 检查可用节点数
2. 可用节点数 **小于** 配置要求时，vLLM 无法完成模型切分和加载
3. 所有节点都加入 Ray 集群后，vLLM 才开始分发模型参数并加载

💡 AI 补充

整个启动过程实际上是一个分阶段递进的过程，从 Ray 集群初始化到服务最终就绪，需要依次完成四个步骤。

1️⃣ Ray 集群初始化：Head 启动，等待 Worker 加入

2️⃣ vLLM 服务初始化：检查 Ray 集群中的 GPU 数量

3️⃣ 模型切分与加载：GPU 数满足要求后按层切分

4️⃣ 服务就绪：状态变为 `started`，可接收推理请求

因此，启动 Head 节点后看到“阻塞等待”是 **正常现象**，而非错误。这是集群启动流程中一个必经的中间状态。

阻塞 ≠ 卡死：初学者容易误以为服务卡死。实际上，vLLM 在等待 Worker 节点时会在日志中输出类似 `Waiting for workers to join...` 的提示。

并行参数必须匹配节点数：`pipeline_parallel_size` 必须等于节点数（本案例为 2）。如果设置为 1，则不需要等待 Worker 节点，但无法实现跨节点流水线并行。

显存未占用是正常状态：在 Worker 节点加入之前，模型尚未开始加载，因此显存为空。这是验证“等待状态”的一个直观指标。

本节小结

- `start_head.sh` 脚本负责环境变量配置、Ray Head 节点启动和 vLLM 服务启动，是集群启动的核心入口。
- Head 节点的 IP 是 WSL 内部 IP（桥接模式下为局域网 IP），Worker 节点将通过此 IP 连接；必须使用桥接模式。
- vLLM 配置流水线并行（`pipeline_parallel_size=2`），必须等待 Worker 节点加入后才能加载模型，因此启动后进入阻塞等待状态是正常现象。
- 下一步（B3）将在主机 B 上启动 Worker 节点，连接 Head 节点并加入 Ray 集群，届时两个 GPU 将被识别，模型加载将正式开始。

B3. Worker节点启动：连接Head节点并加入Ray集群

引入

章前总览： 在 B2 中，主机 A 已作为 Head 节点完成了 vLLM 服务的初始化，但由于采用**流水线并行（Pipeline Parallelism）**且集群规模为双节点，vLLM 服务阻塞在等待 Worker 加入的状态。本章的任务是：在主机 B 上完成 Worker 节点的启动，将其接入由 Head 节点创建的 Ray 集群，从而使分布式推理集群真正"成形"。可以说，B2 搭好了"骨架"，B3 才让集群拥有了"第二条腿"。

逻辑链条：

- 启动前环境确认：在主机 B 上确认 WSL 与虚拟交换机配置，进入 WSL Ubuntu 系统
- 启动 Worker 节点：执行 `start_worker.sh` 并指定 Head 节点 IP（如 4.0.0.18）
- 验证集群状态：通过 `ray status` 查看集群节点与资源情况
- 确认 GPU 资源：确认两个节点的 GPU 均被 Ray 正确识别，为 B4 的模型加载做准备

C1. Worker 节点脚本与 Head 节点 IP 的指定

在主机 B 上启动 Worker 节点的操作流程与 Head 节点类似，同样需要先完成环境确认，再执行专用脚本。

操作流程

1. **确认 WSL 环境** — 确认已安装并可进入（如 Ubuntu 22.04）
2. **检查虚拟交换机配置** — 通过 PowerShell 查询桥接模式
3. **核对 .wslconfig 文件** — 确认虚拟交换机及桥接模式定义
4. **进入 WSL 并执行脚本** — 切换到脚本目录，执行 `start_worker.sh`

WSL 环境

确认 WSL 已安装并可进入，建议使用 Ubuntu 22.04 版本，与主机 A 保持版本一致。

虚拟交换机

通过 `Get-VMNetworkAdapter` / `Get-VMSwitch` 确认桥接模式配置正确。

执行启动脚本

```
# 进入 WSL Ubuntu 系统后，切换到脚本所在目录
cd /path/to/scripts

# 执行 Worker 节点启动脚本，指定 Head 节点 IP
bash start_worker.sh --head-ip 4.0.0.18
```

💡 AI 补充

为什么在 Worker 节点上也要重复这些"看似冗余"的确认步骤？因为 Ray 集群的节点发现依赖于底层网络层的互通性。

🔗 桥接模式不一致 → 即使 IP 正确，TCP 连接仍会失败

⚡ GCS 通信、对象传输、NCCL 通信都建立在此网络层之上

因此这些确认步骤不是形式主义，而是故障排查的第一道防线。

🎯 先排查底层网络，再排查上层服务 —— 从内到外的诊断思路

C2. 集群状态验证：ray status 命令的使用

Worker 节点执行启动脚本后，需要通过 Ray 提供的状态命令来验证集群是否真正"成形"。

验证路径

执行 ray status → 查看节点列表 → 确认资源就绪

验证命令

```
ray status
```

节点状态

集群中存在 **2 个节点**，Head 与 Worker 均显示为 Alive 状态。

GPU 资源

两个节点的 GPU 均被 Ray 正确识别并纳入资源池。

若 `ray status` 只显示 1 个节点，说明 Worker 未成功加入。常见原因：Head 节点 IP 填写错误、防火墙拦截了 GCS 端口（默认 6379）、或两台主机不在同一网段。

C3. GPU 资源确认与后续衔接

集群状态验证通过后，还需确认两个节点的 GPU 资源均被 Ray 正确识别，这是 B4 模型加载能够顺利进行的先决条件。

资源确认清单

- Head 节点（主机 A）GPU 已注册到 Ray 资源池
- Worker 节点（主机 B）GPU 已注册到 Ray 资源池
- 集群具备双节点流水线并行的硬件条件

衔接到 B4

至此集群"第二条腿"已就位。B4 中 Head 节点会将模型信息同步到 Worker 节点，两个节点同时开始加载模型，进入跨节点流水线并行阶段。

本节小结

- 主机 B 需与主机 A 对称地完成 WSL 环境与虚拟交换机配置确认，确保底层网络互通，这是 Ray 集群节点发现的前提。
- 执行 `start_worker.sh` 时须正确指定 Head 节点 IP（如 4.0.0.18），Worker 才能通过 Ray 的 GCS 与 Head 建立连接。
- 通过 `ray status` 验证集群节点数与 GPU 资源，确认两个节点均已成功加入，为 B4 模型同步加载做好准备。

B4. 模型加载与同步：跨节点流水线并行

引入

章前总览： 在 B2 中，Head 节点已启动 vLLM 服务并因流水线并行配置而阻塞等待；在 B3 中，Worker 节点已成功通过 Ray 加入集群，`ray status` 确认了两个节点、两块 GPU 均在线。至此，集群的“物理连接”已完成。但**连接 ≠ 可用**——模型权重还只存在于 Head 节点的磁盘上，Worker 节点的 GPU 显存中空空如也。本章要解决的核心问题是：**模型如何从 Head 节点“流”到 Worker 节点，两个节点如何协同完成加载，以及我们如何确认这一切真的发生了？**

逻辑链条：

- Head 节点感知到 Worker 加入后，如何将模型信息传递给 Worker？—— (C1) 模型信息同步机制
- 两个节点各自如何加载模型？显存如何变化？—— (C2) 跨节点模型加载与 GPU 显存监控
- 我们通过什么工具、什么端口、看什么指标来确认集群正常工作？—— (C3) Ray dashboard——端口 8625 与集群状态可视化

衔接提醒： 本章是 B3（集群组建）与 B5（推理验证）之间的**关键过渡**。如果没有完成模型加载与同步，B5 中的推理请求将无从执行。因此，本章既是 B3 的“验收环节”，也是 B5 的“前置条件”。

C1. 模型信息同步机制——Head 节点如何“通知” Worker 节点?

当 Worker 节点通过 `ray start --address=<Head_IP>:<port>` 成功加入 Ray 集群后，Ray 的**分布式对象存储（Distributed Object Store）**和**全局控制存储（Global Control Store, GCS）**会自动感知到新节点的加入。整个同步过程是**自动触发**的，不需要手动干预。

① Ray 感知新节点 → **② vLLM 收到通知** → **③ 调度器触发加载** → **④ 按层分片下发**

📡 **Head 节点 vLLM 进程**

因流水线并行而阻塞等待，收到 Ray 通知后确认集群 GPU 资源已足够，随即开始加载配置信息。

📣 **流水线并行调度器**

将模型名称、层数、张量并行/流水线并行配置、dtype 等信息通过 Ray 通信通道广播给所有 Worker 节点。

🧩 **权重来源与分片**

Head 节点加载前若干层，Worker 节点加载后续层。权重文件（safetensors 或 bin 格式）位于 Head 节点，Worker 通过网络拉取对应分片。

💡 AI 补充

为什么 Worker 节点不自己下载模型，而是从 Head 节点拉取？这背后有三层设计考量。

💡 **一致性保证**：分布式推理要求所有节点加载的模型版本、权重分片严格一致，避免缓存或版本差异。

⚡ **网络拓扑优化**：局域网内从 Head 节点拉取比外网下载更快、更稳定。

🎯 **vLLM 设计逻辑**：Head 节点充当“模型源”，Worker 节点作为“消费者”获取权重分片，这是 Ray 生态中常见的**主从式数据分发**模式。

C2. 跨节点模型加载与 GPU 显存监控——显存为何“双线上升”？

在**流水线并行（Pipeline Parallelism）**模式下，模型按层切分。假设模型有 N 层，流水线并行度为 2（本案例中两个节点），则 Head 节点（GPU 0）负责加载第 1 层到第 N/2 层，Worker 节点（GPU 1）负责加载第 N/2+1 层到第 N 层。因此，两个节点的 GPU 显存**都会上升**，但幅度可能不同。

阶段一 **Worker 加入前**

Head 节点已占用部分显存（因 vLLM 初始化），Worker 节点空载，Head 节点因等待 Worker 而阻塞。

阶段二 **Worker 加入后**

两个节点**同时**从 Head 节点拉取各自负责的权重分片，显存开始上升。

阶段三 **加载过程中**

两个节点显存**逐渐升高**，可通过 Ray dashboard 实时观察上升曲线。

阶段四 **加载完成**

显存稳定在高位，vLLM 服务状态变为 `started/completed`，表示服务已就绪。

💡 AI 补充

为什么两个节点的显存**同时**上升，而不是先 Head 后 Worker？这与 vLLM 的加载方式有关。

> 🔗 **并行拉取**：Head 节点读本地文件，Worker 节点通过网络读 Head 节点的文件，两者几乎同时开始。若只有一方上升，可能意味着网络带宽不足、分片不均衡或配置错误。

可通过以下三个现象快速判断加载是否正常：

> 📊 两端显存曲线同步上扬

> 🟢 状态最终变为 started/completed

易错点 1：认为“Worker 节点显存上升是因为它在运行推理”。
纠正：此时只是**加载模型**，尚未进行推理。显存上升是权重加载的结果，不是计算的结果。

易错点 2：认为“两个节点显存必须一模一样”。
纠正：流水线并行下，两个节点加载的层数可能不同，显存占用**允许有差异**，只要总显存不超过各 GPU 容量即可。

易错点 3：认为“模型加载完成后才能启动 vLLM 服务”。
纠正：vLLM 服务状态 `started/completed` 是加载完成的标志。在此之前，模型加载已在后台进行。

C3. Ray dashboard——端口 8625 与集群状态可视化

Ray Dashboard 是 Ray 集群的**Web 可视化监控界面**，可以直观查看节点状态、GPU 利用率、任务进度、日志等信息。它是我们确认“模型真的加载成功了”的**核心观测窗口**。

🌐 **访问地址**

`http://<Head_IP>:8265`（默认端口）

📡 **端口说明**

材料口述为 8625，但 Ray 标准默认端口是 **8265**，以 `ray start` 输出为准。

8265	VS	8625
Ray 标准默认端口		材料口述的端口

若确实需使用 8625，可在启动脚本中显式指定 `--dashboard-port=8625`。

💡 **AI 补充**

在 dashboard 中，建议重点关注以下四个面板，快速判断集群健康状况。

- 📌 **Nodes 面板**：确认两个节点均为 Alive 状态。
- 📊 **Resources 面板**：查看两 GPU 显存占用是否同步上升。
- ⚙️ **Actors 面板**：确认 vLLM 的 Actor 已分布到两个节点。
- 📝 **Logs 面板**：查看加载日志，定位可能的错误。

本节小结

- Worker 加入 Ray 集群后，模型信息同步**自动触发**：Head 节点将配置广播给 Worker，权重从 Head 节点按层分片拉取，形成主从式数据分发。
- 流水线并行下两个节点的 GPU 显存**几乎同时上升**，但分配层数可能不同，允许存在差异，只要不超过各 GPU 容量即可。
- Ray Dashboard 是核心监控窗口，端口以 `ray start` 输出为准（默认 8265，材料口述为 8625），需重点关注 Nodes、Resources、Actors、Logs 四个面板。
- vLLM 服务状态变为 `started/completed` 是模型加载完成的标志，也是进入 B5 推理验证的前提条件。

B5. 推理验证与集群监控：端到端测试与GPU利用率观察

引入

章前总览： 经过 B1~B4 的层层推进，我们已经完成了双主机 WSL 环境配置、Head 节点启动、Worker 节点加入以及模型跨节点同步加载。此刻，两个节点的 GPU 显存已被模型占满，vLLM 服务状态为 **`started/completed`**。但这一切只证明了"集群搭起来了"，尚未证明"集群能干活"。本章的核心任务是：**发起一次真实的推理请求，验证双节点集群的端到端推理能力，并通过 Ray dashboard 观察推理过程中两个节点 GPU 利用率的动态变化，从而确认流水线并行真正在协同工作。**

逻辑链条：

- 如何发起推理请求：Gradio WebUI 脚本的修改与运行，启动测试客户端
- 请求发出后集群内部发生了什么：请求分发与流水线并行推理机制，Head 协调 Worker 共同 decode
- 如何判断推理是否真正在双节点上协同执行：GPU 利用率监控，decode 时利用率上升、文本越长占用越高、推理结束后回落

C1. Gradio WebUI 脚本——IP 地址修改与端口 8000 配置

要验证集群的推理能力，需要一个客户端来向 vLLM 服务发送请求。这里使用的工具是 **Gradio**——一个快速构建机器学习 Web 界面的 Python 库。

核心概念：Gradio WebUI

Gradio 是一个用于快速创建机器学习演示界面的 Python 库。在本场景中，它充当**推理客户端**的角色：提供一个网页聊天界面，用户输入问题后，Gradio 将请求发送至 vLLM 服务的 API 端点（端口 8000），并展示返回的生成结果。

操作步骤

1 激活环境

进入之前创建的 conda 环境（如果未创建，只需确保安装了 `gradio` 依赖即可： `pip install gradio` ）。

2 修改脚本

找到单机部署时已使用过的 `chat_vllm.py` 脚本（或类似的 Gradio 测试脚本），做两处关键修改：

IP 地址

改为 Head 节点的 IP 地址（如 `4.0.0.18`），因为 vLLM 服务运行在 Head 节点上，API 端点由 Head 节点暴露。

端口号

保持 `8000`（vLLM 默认 API 端口）。

3 运行脚本

执行 `python chat_vllm.py` ，脚本会自动启动一个浏览器页面（Gradio 界面）。

4 远程访问

将浏览器地址复制下来，在另一台机器上打开浏览器访问该地址（方便同时观察测试界面和 Ray dashboard）。

- **IP 地址该填谁的？** 填 **Head 节点**的 IP，不是 Worker 节点的。因为 vLLM 的 API 服务（端口 8000）是在 Head 节点上启动的，Worker 节点只是通过 Ray 加入集群、参与计算，并不直接对外暴露推理 API。
- **端口号为什么是 8000？** 8000 是 vLLM 的默认 API 端口，在 `start_head.sh` 启动 vLLM 服务时已指定。如果启动脚本中改了端口，这里也要对应修改。
- **Gradio 界面中的 API Key 和模型名称：** 在本场景中，Key 不需要填写（本地部署无鉴权），模型名称保持默认即可（即启动 vLLM 时指定的模型）。

C2. 请求分发与流水线并行推理机制

当我们在 Gradio 界面中输入问题并点击发送后，背后发生了什么？这是理解整个集群协同工作的关键。

推理请求的完整链路

1. 用户在 Gradio 界面输入问题
   前端聊天窗口接收用户输入文本
2. Gradio 将请求发送至 vLLM API 端点
   请求通过 HTTP 发送到 Head 节点的 8000 端口
3. Head 节点接收请求，协调 Worker 节点
   vLLM 调度器将请求分发给两个节点的 GPU
4. 双节点协同生成文本
   两个节点的 GPU 同时参与 decode 过程，结果返回给 Gradio 界面

核心机制

⚡

流水线并行

两个节点的 GPU 通过流水线并行协同工作，各承担部分计算负载，共同完成一次完整的文本生成。

🔗

Ray 调度

Ray 作为分布式调度框架，负责将请求分发给各节点 GPU 并协调同步，是跨节点协同的基础。

C3. GPU 利用率监控——验证双节点协同执行

通过 Ray dashboard（Head 节点 IP:8625 端口）可以实时观察两个节点 GPU 利用率的变化。这是判断推理是否真正在双节点上协同执行的**关键证据**。

监控要点

1. **推理时 GPU 利用率上升** — 说明计算请求已送达两个节点的 GPU。
2. **输出文本越长，占用越高** — 生成的 token 数量越多，GPU 持续计算的时间越长。
3. **推理结束后利用率逐渐回落** — 生成完成，GPU 空闲下来，验证了协同推理的闭环。

AI 补充

为什么观察 GPU 利用率能判断流水线并行是否生效？以下是几个关键认知点：

> **若仅单节点工作**：只有 Head 节点的 GPU 利用率会上升，Worker 节点保持空闲。

> **若两节点均上升**：说明流水线并行生效，请求被正确分发到两个 GPU。

因此，Ray dashboard 的 GPU 利用率曲线是验证集群协同推理是否成功的**最直观证据**。

> Ray dashboard 访问地址：`Head节点IP:8625`

本节小结

- Gradio WebUI 是推理客户端，修改脚本中的 IP 为 Head 节点地址、端口保持 8000，即可发起端到端推理请求。
- 请求发出后，Head 节点接收请求并通过 Ray 调度，协调两个节点的 GPU 通过流水线并行共同完成文本生成。
- 通过 Ray dashboard 观察 GPU 利用率变化是验证协同推理的关键：推理时两节点利用率上升，文本越长占用越高，结束后回落。